Summary of API Design Decisions and Logic

The API was designed to be intuitive, RESTful, and flexible for integration into various frontends like web apps or mobile interfaces. Key design choices include:

- **Modular endpoints** for different recommendation strategies (User-User, Item-Item, Content-Based), making it easy to compare and switch between them.
- **Query parameters** (e.g., top_n, title) allow fine-tuned control over outputs.
- **Separation of concerns**: Recommendation logic, book metadata, and authentication are handled through clearly defined endpoints.
- **Lightweight JSON responses** to ensure low-latency communication and better frontend rendering.

Authentication Method and Reasoning

JWT (JSON Web Token) was chosen for authentication due to its simplicity, statelessness, and scalability. Here's why:

- **Stateless sessions**: No need to store sessions on the server, improving scalability.
- **Security**: Tokens are signed, tamper-proof, and can be set to expire for added protection.
- **Ease of integration**: JWTs are easy to work with on the frontend (e.g., in headers or localStorage) and backend.
- **Custom Claims**: Tokens can optionally hold user roles or permissions, enabling role-based access in future expansions.

Deployment Challenges and How We Tackled Them

1. Slow Model Loading

- **Challenge:**
  The recommendation models, particularly the User-User and Item-Item collaborative filtering ones, were quite large in size. Loading them every time the API was triggered led to noticeable delays and poor response times.

- **Solution:**
  We used joblib to serialize (save) the models after training and loaded them only once when the Flask application starts. This means the models stay in memory and are instantly accessible when API calls come in. This significantly reduced latency and improved the user experience.

2. Cold Start Problems for New Users

- **Challenge:**
  Collaborative filtering depends on users having interaction history (like book ratings). For brand-new users, there is no such history, so these models couldn't generate relevant recommendations.
- **Solution:**
  We implemented a fallback mechanism. If a user has no historical data:
  - We use **content-based filtering**, leveraging book metadata (genre, author, etc.) to suggest similar books.
  - Alternatively, we show a list of **popular/trending books** to help users get started and begin generating interaction data.

3. Handling Traffic Spikes & Concurrency

- **Challenge:**
  During load testing and simulations of multiple users accessing the API simultaneously, we noticed sluggishness and inconsistent responses. Flask by itself isn't designed to handle high-concurrency workloads.
- **Solution:**
  We containerized the entire app using **Docker**, making it easier to deploy consistently on any server. For production, we ran the Flask app using **Gunicorn** (a production-grade WSGI server) and added **Nginx** as a reverse proxy to handle more concurrent connections efficiently. This setup improved reliability, scalability, and performance under load.

4. Environment Setup Issues

- **Challenge:**
  Developers using different machines and operating systems ran into version mismatches, missing dependencies, or setup errors that slowed down collaboration and testing.
- **Solution:**
  We standardized the environment using:
  - A requirements.txt file that clearly defines all Python dependencies.
  - Docker for environment replication. With a Docker image, anyone can run the app locally or in production with just a single command (docker-compose up), regardless of their OS or environment.

5. CORS Issues During Frontend Integration

- **Challenge:**
  When we connected the frontend (running on a different port or domain) to the Flask backend, we ran into Cross-Origin Resource Sharing (CORS) errors. The browser was blocking requests due to security policies.
- **Solution:**
  We used the flask-cors package, which enabled the backend to accept requests from different origins (like our frontend server). With this middleware properly configured, the frontend and backend communicated smoothly without security issues.